МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Управление процессами и обмен данными через каналы
Вариант 16

Выполнил: Барышников Е. А.
Группа: М80-201БВ-25
Преподаватель: Миронов Е. С.

Москва, 2026

Условие

Цель работы: приобретение практических навыков управления процессами в ОС и организации обмена данными между процессами посредством каналов.

Задание: составить программу на языке Си, в которой основной процесс создаёт дочерний процесс. Родительская и дочерняя части должны быть отдельными программами. Пользователь первой строкой вводит имя выходного файла, затем строки произвольной длины. Родитель передаёт строки в первый канал. Дочерний процесс проверяет строки: подходящие записывает через стандартный вывод в файл, а сообщения об ошибках передаёт во второй канал. Родитель выводит полученные сообщения в свой стандартный вывод. Необходимо обрабатывать системные ошибки.

Вариант: 16. Строка должна оканчиваться на точку «.» или точку с запятой «;».

Метод решения

Создаются два неименованных канала и один дочерний процесс. Первый канал служит для передачи данных от родителя ребёнку, второй — для ответов ребёнка родителю. Для запуска отдельной дочерней программы используются `fork` и `execl`. Вызов `fork` создаёт процесс, а `execl` заменяет исполняемую программу в уже созданном процессе, сохраняя его PID.

До запуска программы `child` родительская программа в дочерней ветви перенаправляет стандартные дескрипторы:

Поток дочерней программы	Источник или назначение
`stdin`, дескриптор 0	Читающий конец первого канала
`stdout`, дескриптор 1	Выходной файл
`stderr`, дескриптор 2	Пишущий конец второго канала

Родитель сохраняет свои стандартные потоки для взаимодействия с пользователем. Неиспользуемые концы каналов закрываются в обоих процессах.

Обмен выполняется последовательно: родитель отправляет одну строку, сбрасывает буфер записи и ожидает ответ. Ребёнок проверяет последний символ перед переводом строки. Символы `\n` и `\r` при проверке не учитываются; пробелы в конце не удаляются. Пустая строка считается неправильной. Подходящая строка записывается в файл, после чего ребёнок отправляет пустую строку как подтверждение. Для неправильной строки отправляется сообщение с её номером. Служебные подтверждения родитель не показывает.

Чтение выполняется функцией `getline`: буфер увеличивается по мере необходимости, поэтому фиксированного ограничения длины строки нет; фактический предел задаётся доступной памятью. Если последняя введённая строка не имеет перевода строки, родитель добавляет его перед передачей. Порядок строк сохраняется.

После окончания ввода родитель закрывает пишущий конец первого канала. Ребёнок получает EOF, освобождает память и завершает работу. Родитель дочитывает второй канал и вызывает `waitpid`, чтобы получить статус завершения ребёнка и освободить запись о процессе в ядре.

Описание программы

Файл parent.c. Функция `main` читает имя файла, открывает файл, создаёт каналы и дочерний процесс, настраивает перенаправления и ожидает завершения ребёнка. Функция `exchange(int write_fd, int read_fd)` организует передачу строк и чтение ответов. Она возвращает 0 при успешном обмене и 1 при ошибке.

Файл child.c. Функция `main` читает строки из стандартного ввода, проверяет правило варианта, записывает подходящие строки в стандартный вывод и отправляет ответы через стандартный поток ошибок.

Основные типы данных. Файловые дескрипторы имеют тип `int`; это номера открытых ресурсов в таблице процесса. Указатели `FILE *` представляют библиотечные потоки с буферизацией. Функция `fdopen` связывает такой поток с уже открытым дескриптором, не создавая отдельный канал. Тип `pid_t` используется для PID, `size_t` — для размеров буферов, `ssize_t` — для длины прочитанной строки или значения -1 при окончании чтения либо ошибке.

Системные интерфейсы, используемые программой:

Вызов	Назначение
`open`	Открывает файл на запись. Флаги `O_CREAT` и `O_TRUNC` создают отсутствующий файл и очищают существующий. Права 0644 ограничиваются маской `umask`.
`pipe`	Создаёт канал: элемент 0 массива — чтение, элемент 1 — запись.
`fork`	Создаёт ребёнка. Возвращает 0 ребёнку, PID ребёнка родителю или -1 при ошибке.
`dup2`	Переназначает стандартный дескриптор на файл или канал.
`execl`	Загружает `./child` в текущий процесс; при успехе не возвращается.
`close`	Закрывает дескриптор. Закрытие всех пишущих концов позволяет читателю получить EOF после чтения накопленных данных.
`waitpid`	Ожидает конкретный дочерний процесс и получает его статус. При `EINTR` вызов повторяется.
`signal`	Устанавливает игнорирование `SIGPIPE`: при закрытом читающем конце запись может вернуть ошибку вместо завершения процесса этим сигналом.
`_exit`	Завершает дочернюю ветвь при ошибке перенаправления или запуска, не сбрасывая унаследованные буферы библиотеки C.

`getline`, `fdopen`, `fwrite`, `fflush`, `fclose` и `perror` — библиотечные функции. Работа с данными в итоге приводит к обращениям `read` и `write` к ядру. Сброс буфера `fflush` нужен, чтобы другая сторона получила строку до начала ожидания ответа. `fclose` закрывает и поток, и его дескриптор.

Проверяются результаты создания процессов и каналов, открытия файла, перенаправления, запуска программы и операций потокового ввода-вывода. Ошибки выводятся через `perror` или диагностические сообщения. Статус ребёнка проверяется макросами `WIFEXITED` и `WEXITSTATUS`. Неправильная пользовательская строка является результа-

том проверки данных и сама по себе не означает аварийного завершения программы.

Особенность представленной версии: результаты отдельных вызовов `close` при освобождении исходных дескрипторов не проверяются. Поэтому обработка ошибок закрытия реализована для потоков через `fclose`, но не для всех прямых вызовов `close`.

Результаты

Программа собрана как два отдельных исполняемых файла. Для воспроизведения в Linux следует выполнить команды в каталоге, содержащем оба исходных файла:

```
gcc -std=c11 -Wall -Wextra -Werror parent.c -o parent
gcc -std=c11 -Wall -Wextra -Werror child.c -o child
strace -f -tt -yy -s 200 -o strace.txt ./parent
```

Ключ `-f` включает дочерний процесс, `-tt` добавляет время, `-yy` поясняет файловые дескрипторы, `-s 200` задаёт максимальную длину показываемых строк, `-o` сохраняет трассу в файл. Данные вводятся в терминале; завершение ввода — Ctrl+D на пустой строке.

В предоставленной трассировке Ubuntu (AArch64) введено имя файла `result.txt`, затем три строки:

```
Hello.
my boy
bazaru jackson;
```

Вторая строка не соответствует правилу. Родитель вывел сообщение об ошибке для строки 2, а в выходной файл были записаны:

```
Hello.
bazaru jackson;
```

В трассе видны записи 7 и 16 байт в выходной файл. Оба процесса завершились с кодом 0. PID родителя — 8590, PID ребёнка — 8608. Первый канал имеет идентификатор 33590, второй — 33593. Эти номера относятся к данному запуску и при повторении могут измениться.

Имена функций в исходнике и вызовов ядра в трассе могут отличаться из-за реализации библиотеки C и архитектуры системы:

В исходнике	В предоставленном strace
`open`	`openat`
`pipe`	`pipe2`
`fork`	`clone`
`dup2`	`dup3`
`execl`	`execve`
`waitpid`	`wait4`
`signal`	`rt_sigaction`

После закрытия родителем дескриптора записи первого канала чтение у ребёнка возвращает 0. Затем ребёнок закрывает выходной файл и завершается. Родитель получает EOF второго канала и успешно выполняет `wait4` для PID 8608. Это подтверждает передачу данных, работу перенаправлений и корректное завершение в рассмотренном запуске.

Полная трассировка приведена после исходного кода. Отметки `unfinished` и `resumed` показывают, что вызов одного процесса был начат, а его результат зарегистрирован после событий другого процесса. Они сами по себе не являются ошибками.

Репозиторий: `https://github.com/charmy444/OS_lab2`. Пример выше восстановлен по `strace.txt`; находящийся в репозитории `result.txt` сохранён отдельно как пример входных строк, а не как полученный выходной файл.

Выводы

В ходе лабораторной работы реализованы две программы на языке Си, взаимодействующие через два неименованных канала. Освоены создание дочернего процесса, запуск другой программы, перенаправление стандартных потоков и ожидание завершения процесса. Для варианта 16 реализована проверка последнего символа строки: подходящие строки сохраняются в файл, а сообщения об ошибках передаются родителю.

Трассировка Linux показала создание ребёнка, настройку дескрипторов, передачу строк, запись результата и получение статуса завершения. На практике установлено, что для правильного завершения обмена необходимо закрывать неиспользуемые концы каналов, передавать EOF закрытием пишущего конца и своевременно сбрасывать буферы потоков.

Исходная программа

parent.c — родительский процесс

```c
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int exchange(int write_fd, int read_fd)
{
    // объект, через который библиотека C будет работать с этим дескриптором (файл - тип структуры)
    FILE *to_child = fdopen(write_fd, "w");
    if (to_child == NULL) {
        perror("fdopen pipe1");
        close(write_fd);
        close(read_fd);
        return 1;
    }

    // аналогично
    FILE *from_child = fdopen(read_fd, "r");
    if (from_child == NULL) {
        perror("fdopen pipe2");
        fclose(to_child);
        close(read_fd);
        return 1;
    }

    char *line = NULL; // строка с записью
    char *reply = NULL; // ответ ребенка
    size_t line_capacity = 0, reply_capacity = 0;
    ssize_t length;
    int result = 0;

    while ((length = getline(&line, &line_capacity, stdin)) != -1) {
        // добавляем разделитель во втором условии, если последняя строка пришла без enter
        if (fwrite(line, 1, (size_t)length, to_child) != (size_t)length ||
            (line[length - 1] != '\n' && fputc('\n', to_child) == EOF) ||
            fflush(to_child) == EOF) {
            perror("write pipe1");
            result = 1;
            break;
        }

        ssize_t reply_length = getline(&reply, &reply_capacity, from_child);
        if (reply_length == -1) {
            if (feof(from_child)) // конец потока
                fprintf(stderr, "дочерняя программа завершилась без ответа\n");
            else // не конец потока
                perror("read pipe2");
            result = 1;
            break;
        }

        // пустой ответ означает успех, остальные ответы выводим пользователю
        if (reply_length > 1 &&
            (fwrite(reply, 1, (size_t)reply_length, stdout) != (size_t)reply_length ||
             fflush(stdout) == EOF)) {
            perror("write console");
            result = 1;
            break;
        }
    }

    // ошибка при чтении, символ окончания строки не обнаружен
    if (length == -1 && !feof(stdin)) {
        perror("read stdin");
        result = 1;
```

```
70      }
71
72      // закрытие pipe1 сообщает ребёнку, что новых строк больше не будет
73      if (fclose(to_child) == EOF) {
74          perror("close pipe1");
75          result = 1;
76      }
77
78      // забираем возможные сообщения об ошибках при завершении ребёнка
79      while ((length = getline(&reply, &reply_capacity, from_child)) != -1) {
80          if (length > 1 &&
81              fwrite(reply, 1, (size_t)length, stdout) != (size_t)length) {
82              perror("write console");
83              result = 1;
84              break;
85          }
86      }
87
88      // аналагично проверяем как с каналоми родителя
89      if (length == -1 && !feof(from_child)) {
90          perror("read pipe2");
91          result = 1;
92      }
93      if (fclose(from_child) == EOF) {
94          perror("close pipe2");
95          result = 1;
96      }
97      // сбрасываем буфер так как в вайле не вызывали сброс
98      if (fflush(stdout) == EOF) {
99          perror("flush console");
100         result = 1;
101     }
102     free(line);
103     free(reply);
104     return result;
105 }
106
107 int main(void)
108 {
109     // вместо аварийного завершения получим ошибку записи
110     if (signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
111         perror("signal");
112         return 1;
113     }
114
115     char *filename = NULL;
116     size_t capacity = 0;
117     ssize_t length = getline(&filename, &capacity, stdin);
118     if (length == -1) {
119         if (feof(stdin))
120             fprintf(stderr, "не введено имя файла\n");
121         else
122             perror("read filename");
123         free(filename);
124         return 1;
125     }
126     if (length > 0 && filename[length - 1] == '\n')
127         filename[--length] = '\0';
128     if (length > 0 && filename[length - 1] == '\r')
129         filename[--length] = '\0';
130
131     int file = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
132     free(filename);
133     if (file == -1) {
134         perror("open");
135         return 1;
136     }
137
138     int pipe1[2], pipe2[2];
139     if (pipe(pipe1) == -1) {
140         perror("pipe1");
141         close(file);
142         return 1;
```

```
143     }
144     if (pipe(pipe2) == -1) {
145         perror("pipe2");
146         close(file);
147         close(pipe1[0]);
148         close(pipe1[1]);
149         return 1;
150     }
151 
152     pid_t pid = fork();
153     if (pid == -1) {
154         perror("fork");
155         close(file);
156         close(pipe1[0]);
157         close(pipe1[1]);
158         close(pipe2[0]);
159         close(pipe2[1]);
160         return 1;
161     }
162 
163     if (pid == 0) {
164         if (dup2(pipe2[1], STDERR_FILENO) == -1 ||
165             dup2(pipe1[0], STDIN_FILENO) == -1 ||
166             dup2(file, STDOUT_FILENO) == -1) {
167             perror("dup2");
168             _exit(1);
169         }
170         close(pipe1[0]);
171         close(pipe1[1]);
172         close(pipe2[0]);
173         close(pipe2[1]);
174         close(file);
175 
176         execl("./child", "child", (char *)NULL);
177         perror("execl");
178         _exit(1);
179     }
180 
181     close(pipe1[0]);
182     close(pipe2[1]);
183     close(file);
184     int result = exchange(pipe1[1], pipe2[0]);
185 
186     int status;
187     pid_t waited;
188     do {
189         waited = waitpid(pid, &status, 0);
190     } while (waited == -1 && errno == EINTR); // повторяем пока прерывается сигналом
191 
192     if (waited == -1) {
193         perror("waitpid");
194         return 1;
195     }
196     if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
197         fprintf(stderr, "дочерняя программа завершилась с ошибкой\n");
198         return 1;
199     }
200     return result;
201 }
```

child.c — дочерний процесс

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *line = NULL;
    size_t capacity = 0;
    ssize_t length;
    size_t number = 0;
    int result = 0;

    while ((length = getline(&line, &capacity, stdin)) != -1) {
        number++;
        ssize_t end = length;

        if (end > 0 && line[end - 1] == '\n')
            end--;
        if (end > 0 && line[end - 1] == '\r')
            end--;

        if (end > 0 && (line[end - 1] == '.' || line[end - 1] == ';')) {
            // stdout уже перенаправлен родителем в файл
            if (fwrite(line, 1, (size_t)length, stdout) != (size_t)length ||
                fflush(stdout) == EOF) {
                perror("write file");
                result = 1;
                break;
            }
            // служебное подтверждение, одна пустая строка
            if (fputc('\n', stderr) == EOF) {
                result = 1;
                break;
            }
        } else {
            if (fprintf(stderr,
                        "ошибка - строка %zu должна оканчиваться на '.' или ';'\n",
                        number) < 0) {
                result = 1;
                break;
            }
        }
        if (fflush(stderr) == EOF) {
            result = 1;
            break;
        }
    }

    if (length == -1 && !feof(stdin)) {
        perror("read stdin");
        result = 1;
    }
    free(line);
    if (fclose(stdout) == EOF) {
        perror("close file");
        result = 1;
    }
    return result;
}
```

Трассировка системных вызовов

Ниже приведён полный журнал предоставленного запуска без изменения содержимого.

```
8590  02:55:32.288225 execve("./parent", ["./parent"], 0xffffdeb1bc48 /* 54 vars */) = 0
8590  02:55:32.290460 brk(NULL)         = 0xc729e9ad8000
8590  02:55:32.290896 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf867b8e37000
8590  02:55:32.292529 faccessat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "/etc/ld.so.preload", R_OK) = -1
↪  ENOENT (No such file or directory)
8590  02:55:32.293053 openat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC)
↪  = 3</etc/ld.so.cache>
8590  02:55:32.293634 fstat(3</etc/ld.so.cache>, {st_mode=S_IFREG|0644, st_size=72223, ...}) = 0
8590  02:55:32.294502 mmap(NULL, 72223, PROT_READ, MAP_PRIVATE, 3</etc/ld.so.cache>, 0) = 0xf867b8def000
8590  02:55:32.294774 close(3</etc/ld.so.cache>) = 0
8590  02:55:32.294975 openat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "/usr/lib/aarch64-linux-gnu/libc.so.6",
↪  O_RDONLY|O_CLOEXEC) = 3</usr/lib/aarch64-linux-gnu/libc.so.6>
8590  02:55:32.295041 read(3</usr/lib/aarch64-linux-gnu/libc.so.6>, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\2
↪  67\0\1\0\0\0\3101\2\0\0\0\0\0@\0\0\0\0\0\0\0P9\33\0\0\0\0\0\0\0\0@\08\0\r\0@\0@\0?\0\6\0\0\0\4\0\0\0
↪  @\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\330\2\0\0\0\0\0\0\330\2\0\0\0\0\0\0\10\0\0\0\0\0\0\0\3\0
↪  \0\0\4\0\0\0\250?\27\0\0\0\0\0\250?\27\0\0\0\0\0\250?\27\0\0\0\0\0\33\0\0\0\0\0\0\0\33\0\0\0\0\0\0\0\1
↪  0\0\0\0\0\0\0\0\1\0\0\0\5\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
↪  832
8590  02:55:32.295136 fstat(3</usr/lib/aarch64-linux-gnu/libc.so.6>, {st_mode=S_IFREG|0755,
↪  st_size=1788240, ...}) = 0
8590  02:55:32.295179 mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
↪  0xf867b8c11000
8590  02:55:32.295225 mmap(0xf867b8c20000, 1891904, PROT_READ|PROT_EXEC,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3</usr/lib/aarch64-linux-gnu/libc.so.6>, 0) = 0xf867b8c20000
8590  02:55:32.295284 munmap(0xf867b8c11000, 61440) = 0
8590  02:55:32.295312 munmap(0xf867b8dee000, 3648) = 0
8590  02:55:32.295333 mprotect(0xf867b8dc9000, 81920, PROT_NONE) = 0
8590  02:55:32.295364 mmap(0xf867b8ddd000, 20480, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3</usr/lib/aarch64-linux-gnu/libc.so.6>, 0x1ad000) =
↪  0xf867b8ddd000
8590  02:55:32.295535 mmap(0xf867b8de2000, 48704, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xf867b8de2000
8590  02:55:32.295683 close(3</usr/lib/aarch64-linux-gnu/libc.so.6>) = 0
8590  02:55:32.295938 set_tid_address(0xf867b8e38388) = 8590
8590  02:55:32.296064 set_robust_list(0xf867b8e38040, 24) = 0
8590  02:55:32.296222 rseq(0xf867b8e38740, 0x21, 0, 0xd428bc00) = 0
8590  02:55:32.296612 mprotect(0xf867b8ddd000, 12288, PROT_READ) = 0
8590  02:55:32.296653 mprotect(0xc729b469f000, 4096, PROT_READ) = 0
8590  02:55:32.296676 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf867b8e35000
8590  02:55:32.296717 mprotect(0xf867b8e3f000, 8192, PROT_READ) = 0
8590  02:55:32.296781 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
8590  02:55:32.296864 getrandom("\x3d\x06\xa7\x57\xb6\xcb\xc1\x18", 8, GRND_NONBLOCK) = 8
8590  02:55:32.296973 munmap(0xf867b8def000, 72223) = 0
8590  02:55:32.297220 rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[],
↪  sa_flags=SA_INTERRUPT|SA_NODEFER|SA_RESETHAND}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
8590  02:55:32.297474 brk(NULL)         = 0xc729e9ad8000
8590  02:55:32.297687 brk(0xc729e9c00000) = 0xc729e9c00000
8590  02:55:32.297838 fstat(0</dev/pts/0<char 136:0>>, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0),
↪  ...}) = 0
8590  02:55:32.298127 read(0</dev/pts/0<char 136:0>>, "result.txt\n", 1024) = 11
8590  02:55:45.388672 openat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "result.txt", O_WRONLY|O_CREAT|O_TRUNC,
↪  0644) = 3</home/ubuntu/Desktop/lab2/result.txt>
8590  02:55:45.389110 pipe2([4<pipe:[33590]>, 5<pipe:[33590]>], 0) = 0
8590  02:55:45.389260 pipe2([6<pipe:[33593]>, 7<pipe:[33593]>], 0) = 0
8590  02:55:45.389459 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
8590  02:55:45.389717 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD
↪  <unfinished ...>
8608  02:55:45.390304 set_robust_list(0xf867b8e38040, 24) = 0
8608  02:55:45.390555 rt_sigprocmask(SIG_SETMASK, [] <unfinished ...>
8590  02:55:45.390702 <... clone resumed>, child_tidptr=0xf867b8e38030) = 8608
8608  02:55:45.390732 <... rt_sigprocmask resumed>, NULL, 8) = 0
8590  02:55:45.390806 rt_sigprocmask(SIG_SETMASK, [] <unfinished ...>
8608  02:55:45.390968 dup3(7<pipe:[33593]>, 2</dev/pts/0<char 136:0>>, 0 <unfinished ...>
8590  02:55:45.391035 <... rt_sigprocmask resumed>, NULL, 8) = 0
8590  02:55:45.391134 close(4<pipe:[33590]> <unfinished ...>
8608  02:55:45.391170 <... dup3 resumed>) = 2<pipe:[33593]>
```

```
8608  02:55:45.391948 dup3(4<pipe:[33590]>, 0</dev/pts/0<char 136:0>>, 0 <unfinished ...>
8590  02:55:45.392607 <... close resumed>) = 0
8608  02:55:45.392654 <... dup3 resumed>) = 0<pipe:[33590]>
8590  02:55:45.392761 close(7<pipe:[33593]> <unfinished ...>
8608  02:55:45.392949 dup3(3</home/ubuntu/Desktop/lab2/result.txt>, 1</dev/pts/0<char 136:0>>, 0
↪  <unfinished ...>
8590  02:55:45.393039 <... close resumed>) = 0
8590  02:55:45.393146 close(3</home/ubuntu/Desktop/lab2/result.txt> <unfinished ...>
8608  02:55:45.393184 <... dup3 resumed>) = 1</home/ubuntu/Desktop/lab2/result.txt>
8590  02:55:45.393246 <... close resumed>) = 0
8608  02:55:45.393266 close(4<pipe:[33590]> <unfinished ...>
8590  02:55:45.393387 fcntl(5<pipe:[33590]>, F_GETFL <unfinished ...>
8608  02:55:45.393437 <... close resumed>) = 0
8590  02:55:45.393465 <... fcntl resumed>) = 0x1 (flags O_WRONLY)
8608  02:55:45.393519 close(5<pipe:[33590]>) = 0
8608  02:55:45.393577 close(6<pipe:[33593]>) = 0
8590  02:55:45.393700 fcntl(6<pipe:[33593]>, F_GETFL <unfinished ...>
8608  02:55:45.393730 close(7<pipe:[33593]> <unfinished ...>
8590  02:55:45.393851 <... fcntl resumed>) = 0 (flags O_RDONLY)
8608  02:55:45.394065 <... close resumed>) = 0
8590  02:55:45.394153 read(0</dev/pts/0<char 136:0>> <unfinished ...>
8608  02:55:45.394383 close(3</home/ubuntu/Desktop/lab2/result.txt>) = 0
8608  02:55:45.395115 execve("./child", ["child"], 0xffffffbf0f238 /* 54 vars */) = 0
8608  02:55:45.396004 brk(NULL)         = 0xbc8ed5ebf000
8608  02:55:45.396299 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf0b7cfef3000
8608  02:55:45.396584 faccessat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "/etc/ld.so.preload", R_OK) = -1
↪  ENOENT (No such file or directory)
8608  02:55:45.396995 openat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC)
↪  = 3</etc/ld.so.cache>
8608  02:55:45.397254 fstat(3</etc/ld.so.cache>, {st_mode=S_IFREG|0644, st_size=72223, ...}) = 0
8608  02:55:45.397452 mmap(NULL, 72223, PROT_READ, MAP_PRIVATE, 3</etc/ld.so.cache>, 0) = 0xf0b7cfeab000
8608  02:55:45.397721 close(3</etc/ld.so.cache>) = 0
8608  02:55:45.397974 openat(AT_FDCWD</home/ubuntu/Desktop/lab2>, "/usr/lib/aarch64-linux-gnu/libc.so.6",
↪  O_RDONLY|O_CLOEXEC) = 3</usr/lib/aarch64-linux-gnu/libc.so.6>
8608  02:55:45.398029 read(3</usr/lib/aarch64-linux-gnu/libc.so.6>, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\2
↪  67\0\1\0\0\0\3101\2\0\0\0\0\0@\0\0\0\0\0\0\0P9\33\0\0\0\0\0\0\0\0@\08\0\r\0@\0@\0?\0\6\0\0\0\4\0\0\0
↪  @\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\330\2\0\0\0\0\0\0\330\2\0\0\0\0\0\0\10\0\0\0\0\0\0\0\3\0
↪  \0\0\4\0\0\0\250?\27\0\0\0\0\0\250?\27\0\0\0\0\0\250?\27\0\0\0\0\0\33\0\0\0\0\0\0\0\33\0\0\0\0\0\0\0\1
↪  0\0\0\0\0\0\0\0\1\0\0\0\5\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
↪  832
8608  02:55:45.398062 fstat(3</usr/lib/aarch64-linux-gnu/libc.so.6>, {st_mode=S_IFREG|0755,
↪  st_size=1788240, ...}) = 0
8608  02:55:45.398091 mmap(NULL, 1957440, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
↪  0xf0b7cfccd000
8608  02:55:45.398126 mmap(0xf0b7cfcd0000, 1891904, PROT_READ|PROT_EXEC,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3</usr/lib/aarch64-linux-gnu/libc.so.6>, 0) = 0xf0b7cfcd0000
8608  02:55:45.398178 munmap(0xf0b7cfccd000, 12288) = 0
8608  02:55:45.398204 munmap(0xf0b7cfe9e000, 52800) = 0
8608  02:55:45.398227 mprotect(0xf0b7cfe79000, 81920, PROT_NONE) = 0
8608  02:55:45.398258 mmap(0xf0b7cfe8d000, 20480, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3</usr/lib/aarch64-linux-gnu/libc.so.6>, 0x1ad000) =
↪  0xf0b7cfe8d000
8608  02:55:45.398309 mmap(0xf0b7cfe92000, 48704, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xf0b7cfe92000
8608  02:55:45.398346 close(3</usr/lib/aarch64-linux-gnu/libc.so.6>) = 0
8608  02:55:45.398399 set_tid_address(0xf0b7cfef4388) = 8608
8608  02:55:45.398424 set_robust_list(0xf0b7cfef4040, 24) = 0
8608  02:55:45.398450 rseq(0xf0b7cfef4740, 0x21, 0, 0xd428bc00) = 0
8608  02:55:45.398536 mprotect(0xf0b7cfe8d000, 12288, PROT_READ) = 0
8608  02:55:45.398581 mprotect(0xbc8eae4cf000, 4096, PROT_READ) = 0
8608  02:55:45.398620 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↪  0xf0b7cfef1000
8608  02:55:45.398662 mprotect(0xf0b7cfefb000, 8192, PROT_READ) = 0
8608  02:55:45.398721 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
8608  02:55:45.398775 getrandom("\xee\xfd\xe6\x63\xff\xa6\x11\xcf", 8, GRND_NONBLOCK) = 8
8608  02:55:45.398806 munmap(0xf0b7cfeab000, 72223) = 0
8608  02:55:45.398962 brk(NULL)         = 0xbc8ed5ebf000
8608  02:55:45.398983 brk(0xbc8ed6000000) = 0xbc8ed6000000
8608  02:55:45.399021 fstat(0<pipe:[33590]>, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
8608  02:55:45.399063 read(0<pipe:[33590]> <unfinished ...>
8590  02:55:54.394446 <... read resumed>, "Hello.\n", 1024) = 7
8590  02:55:54.394648 fstat(5<pipe:[33590]>, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
```

```
8590  02:55:54.394967 write(5<pipe:[33590]>, "Hello.\n", 7) = 7
8608  02:55:54.395107 <... read resumed>, "Hello.\n", 4096) = 7
8590  02:55:54.395133 fstat(6<pipe:[33593]>, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
8608  02:55:54.395243 fstat(1</home/ubuntu/Desktop/lab2/result.txt> <unfinished ...>
8590  02:55:54.395290 read(6<pipe:[33593]> <unfinished ...>
8608  02:55:54.395412 <... fstat resumed>, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
8608  02:55:54.395551 write(1</home/ubuntu/Desktop/lab2/result.txt>, "Hello.\n", 7) = 7
8608  02:55:54.395970 write(2<pipe:[33593]>, "\n", 1) = 1
8590  02:55:54.396184 <... read resumed>, "\n", 4096) = 1
8608  02:55:54.396210 read(0<pipe:[33590]> <unfinished ...>
8590  02:55:54.396458 read(0</dev/pts/0<char 136:0>>, "my boy\n", 1024) = 7
8590  02:56:13.488535 write(5<pipe:[33590]>, "my boy\n", 7) = 7
8608  02:56:13.488805 <... read resumed>, "my boy\n", 4096) = 7
8590  02:56:13.488940 read(6<pipe:[33593]> <unfinished ...>
8608  02:56:13.489013 write(2<pipe:[33593]>, "\320\276\321\210\320\270\320\261\320\272\320\260 -
↪  \321\201\321\202\321\200\320\276\320\272\320\260 2 \320\264\320\276\320\273\320\266\320\275\320\260
↪  \320\276\320\272\320\260\320\275\321\207\320\270\320\262\320\260\321\202\321\214\321\201\321\217
↪  \320\275\320\260 '.' \320\270\320\273\320\270 ';'\n", 88 <unfinished ...>
8590  02:56:13.489178 <... read resumed>, "\320\276\321\210\320\270\320\261\320\272\320\260 -
↪  \321\201\321\202\321\200\320\276\320\272\320\260 2 \320\264\320\276\320\273\320\266\320\275\320\260
↪  \320\276\320\272\320\260\320\275\321\207\320\270\320\262\320\260\321\202\321\214\321\201\321\217
↪  \320\275\320\260 '.' \320\270\320\273\320\270 ';'\n", 4096) = 88
8608  02:56:13.489219 <... write resumed>) = 88
8590  02:56:13.489320 fstat(1</dev/pts/0<char 136:0>> <unfinished ...>
8608  02:56:13.489467 read(0<pipe:[33590]> <unfinished ...>
8590  02:56:13.489510 <... fstat resumed>, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
8590  02:56:13.489710 write(1</dev/pts/0<char 136:0>>, "\320\276\321\210\320\270\320\261\320\272\320\260 -
↪  \321\201\321\202\321\200\320\276\320\272\320\260 2 \320\264\320\276\320\273\320\266\320\275\320\260
↪  \320\276\320\272\320\260\320\275\321\207\320\270\320\262\320\260\321\202\321\214\321\201\321\217
↪  \320\275\320\260 '.' \320\270\320\273\320\270 ';'\n", 88) = 88
8590  02:56:13.490081 read(0</dev/pts/0<char 136:0>>, "bazaru jackson;\n", 1024) = 16
8590  02:56:24.819352 write(5<pipe:[33590]>, "bazaru jackson;\n", 16) = 16
8608  02:56:24.819652 <... read resumed>, "bazaru jackson;\n", 4096) = 16
8590  02:56:24.819720 read(6<pipe:[33593]> <unfinished ...>
8608  02:56:24.819922 write(1</home/ubuntu/Desktop/lab2/result.txt>, "bazaru jackson;\n", 16) = 16
8608  02:56:24.820264 write(2<pipe:[33593]>, "\n", 1 <unfinished ...>
8590  02:56:24.820439 <... read resumed>, "\n", 4096) = 1
8608  02:56:24.820482 <... write resumed>) = 1
8590  02:56:24.820574 read(0</dev/pts/0<char 136:0>> <unfinished ...>
8608  02:56:24.820685 read(0<pipe:[33590]> <unfinished ...>
8590  02:56:37.144212 <... read resumed>, "", 1024) = 0
8590  02:56:37.144583 close(5<pipe:[33590]>) = 0
8608  02:56:37.144951 <... read resumed>, "", 4096) = 0
8590  02:56:37.145083 read(6<pipe:[33593]> <unfinished ...>
8608  02:56:37.145174 close(1</home/ubuntu/Desktop/lab2/result.txt>) = 0
8608  02:56:37.145501 exit_group(0)     = ?
8590  02:56:37.146168 <... read resumed>, "", 4096) = 0
8608  02:56:37.146205 +++ exited with 0 +++
8590  02:56:37.146326 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=8608, si_uid=1000,
↪  si_status=0, si_utime=0, si_stime=0} ---
8590  02:56:37.146471 close(6<pipe:[33593]>) = 0
8590  02:56:37.146689 wait4(8608, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 8608
8590  02:56:37.147304 exit_group(0)     = ?
8590  02:56:37.147925 +++ exited with 0 +++
```